DESIGN AND IMPLEMENTATION OF A SMART IOT-BASED DOOR ACCESS MONITORING SYSTEM USING ESP32, FIREBASE, AND STREAMLIT

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The rapid growth of the Internet of Things (IoT) has significantly transformed the way physical systems are monitored, controlled, and integrated with digital infrastructures. IoT technologies enable physical devices such as sensors, embedded systems, and networked controllers to collect data from their environment and communicate that information to cloud-based platforms for analysis and visualization. This capability has created new opportunities in various sectors including smart homes, industrial automation, healthcare monitoring, and security management.

Security monitoring systems, particularly those related to building access control, have increasingly benefited from IoT integration. Traditional door access systems often rely on standalone mechanisms such as mechanical locks, keypad systems, or local authentication devices that lack centralized monitoring capabilities. These conventional systems typically provide limited visibility into access activities and offer little support for real-time data analysis or remote monitoring. As a result, it becomes difficult to track unauthorized access attempts, analyze access patterns, or maintain comprehensive audit logs for security purposes.

IoT-based monitoring systems address these limitations by enabling real-time data collection and centralized logging of access events. Through the use of embedded devices connected to cloud platforms, access events can be captured, stored, and analyzed continuously. This allows administrators or security personnel to remotely monitor access activity, detect unusual patterns, and respond promptly to potential security threats.

Modern IoT architectures typically consist of several interconnected layers including sensing devices, edge processing units, communication networks, cloud data management platforms, and application-layer dashboards. The sensing layer captures physical events, the edge layer performs local data processing, the communication layer transmits data to remote systems, the cloud layer stores and manages data, and the application layer provides visualization and control interfaces.

This layered architecture enables scalable and flexible monitoring systems capable of handling large volumes of data and supporting intelligent decision-making processes.

Within the context of building security, IoT-based door monitoring systems can improve situational awareness by maintaining digital records of access activity and providing analytical insights through monitoring dashboards. Such systems also enable the integration of additional features such as anomaly detection, automated alerts, and historical data analysis.

This project explores the design and implementation of a smart IoT-based door access monitoring system that demonstrates how embedded hardware, cloud infrastructure, and data visualization technologies can be integrated to provide real-time monitoring of door access events.

1.2 Problem Statement

Traditional door access monitoring systems are often limited in functionality and lack the ability to provide real-time visibility into access activities. Many conventional access systems operate as isolated devices that record minimal information and provide limited auditing capabilities. As a result, it becomes difficult for administrators to track access history, analyze usage patterns, or detect suspicious behavior.

In environments where security monitoring is important, the absence of centralized logging and real-time monitoring can create vulnerabilities. Unauthorized access attempts may go unnoticed, and security incidents may only be discovered after they occur. Furthermore, traditional systems rarely provide analytical tools that allow administrators to evaluate system performance or identify trends in access behavior.

With the increasing availability of affordable embedded hardware and cloud-based data platforms, it is now possible to design IoT-based monitoring systems that address these limitations. By integrating sensing devices, wireless communication technologies, and cloud data management platforms, access events can be automatically recorded and analyzed in real time.

Despite these technological advances, many small-scale security systems still lack practical implementations that demonstrate how these components can be effectively integrated into a complete IoT architecture. Therefore, there is a need for a practical system that illustrates how IoT

technologies can be used to monitor door access events, store data in the cloud, and provide real-time visualization through a monitoring dashboard.

This project addresses this challenge by designing and implementing an IoT-based door access monitoring system capable of capturing access events, transmitting them to a cloud database, and presenting the information through an interactive monitoring interface.

1.3 Aim of the Project

The aim of this project is to design and implement a smart IoT-based door access monitoring system that integrates embedded sensing devices, wireless communication technologies, cloud-based data storage, and an interactive monitoring dashboard for real-time visualization and analysis of access events.

1.4 Project Objectives

1.4.1 General Objective

To develop a functional IoT system capable of monitoring door access events by integrating sensing hardware, embedded processing, wireless communication, cloud storage, and data visualization technologies.

1.4.2 Specific Objectives

The specific objectives of the project are:

1. To design an IoT system architecture that incorporates sensing, edge processing, communication, cloud data management, and application layers.
2. To implement an embedded system using the ESP32 microcontroller for detecting and processing door access events.
3. To establish wireless communication between the embedded device and a cloud database using WiFi networking.
4. To store access event data in Firebase Realtime Database for centralized logging and retrieval.

5. To develop a web-based monitoring dashboard using Streamlit for visualizing access activity and system analytics.
6. To implement basic security analytics capable of identifying suspicious patterns such as repeated unauthorized access attempts.

1.5 Scope of the Study

This project focuses on the design and development of a prototype IoT door access monitoring system using simulated access events. The system demonstrates the core functionality of an IoT-based monitoring architecture including event detection, wireless communication, cloud data storage, and dashboard visualization.

The embedded system is implemented using an ESP32 microcontroller, while access events are simulated using hardware input buttons to represent authorized and unauthorized access attempts. Event data is transmitted via WiFi to a Firebase Realtime Database where it is stored for analysis and monitoring.

The monitoring interface is implemented using the Streamlit framework, which retrieves data from the cloud database and displays access statistics, historical logs, and visual analytics through an interactive dashboard.

Although the system demonstrates the fundamental architecture of an IoT access monitoring solution, certain advanced features such as biometric authentication, encrypted communication channels, and mobile integration are outside the scope of this prototype and may be considered for future work.

1.6 Significance of the Study

The development of IoT-based monitoring systems offers several advantages for security management and system analysis. By digitizing access events and storing them in a centralized cloud database, administrators can obtain valuable insights into system usage and security behavior.

The proposed SmartDoor monitoring system demonstrates how low-cost embedded devices and cloud platforms can be combined to create an intelligent monitoring infrastructure. The system enables real-time monitoring of access activity, maintains a digital record of access events, and provides analytical visualizations that support informed decision-making.

From an educational perspective, the project also demonstrates the practical application of IoT system engineering principles including embedded programming, wireless communication, cloud integration, and data visualization. The system therefore serves as an example of how IoT technologies can be applied to address real-world security monitoring challenges.

1.7 Organization of the Report

This report is organized into several chapters to systematically present the design, implementation, and evaluation of the SmartDoor IoT monitoring system. Chapter One introduces the background of IoT-based security monitoring systems, defines the problem addressed by the study, and outlines the objectives and scope of the project. Chapter Two presents the system design and architecture, describing the layered IoT framework used in the implementation and explaining the role of each system component. Chapter Three describes the implementation of the system, including the hardware configuration, firmware development, cloud database integration, and dashboard development. Chapter Four presents the results and evaluation of the system, including system testing procedures, performance measurements, and analysis of the monitoring capabilities. Chapter Five discusses the limitations of the implemented system and proposes possible improvements for future development. Finally, Chapter Six concludes the report by summarizing the achievements of the project and highlighting potential areas for future research and system enhancement.

CHAPTER 2: SYSTEM DESIGN AND ARCHITECTURE

2.1 Introduction

This chapter presents the design and architecture of the SmartDoor IoT Access Monitoring System. The system was designed following a layered Internet of Things (IoT) architecture to ensure modularity, scalability, and efficient integration between hardware and software components. The architecture integrates sensing components, an embedded processing unit, communication mechanisms, cloud-based data management, and an application-layer monitoring dashboard.

The objective of the system design was to develop an end-to-end IoT prototype capable of detecting door access events, transmitting the captured data to a cloud database, and presenting the information through an interactive monitoring interface. The design emphasizes reliability, real-time monitoring capability, and modular separation of system functions.

The SmartDoor monitoring system follows a five-layer IoT architecture consisting of the sensing layer, embedded processing layer, communication layer, data management layer, and application layer. Each layer performs a specific function within the overall system.

2.2 Overall System Architecture

The SmartDoor system architecture is organized into interconnected functional layers that facilitate data flow from physical sensing components to a cloud-based monitoring platform.

The general architecture of the system is illustrated conceptually below:

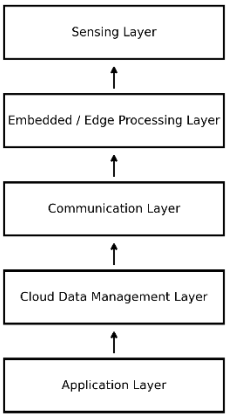

Figure 1 Layered Architecture of the SmartDoor IoT Monitoring System

In this architecture, access events generated at the sensing layer are processed by the embedded microcontroller before being transmitted through a wireless communication network to the cloud database. The stored data is then retrieved and visualized through a monitoring dashboard at the application layer.

This layered design ensures clear separation of responsibilities between system components while enabling efficient data acquisition, processing, storage, and visualization.

2.3 Sensing and Data Acquisition Layer

The sensing layer represents the physical interface between the system and the environment in which access events occur. Its primary function is to detect door access activity and convert physical interactions into digital signals that can be processed by the embedded system.

In the SmartDoor prototype, access events are simulated using hardware push buttons connected to the ESP32 microcontroller. These buttons represent two possible access scenarios:

- Authorized access attempt
- Unauthorized access attempt

Each button press generates a digital input signal that is interpreted by the microcontroller as a door access event.

In addition to the input components, several output devices were incorporated into the system to provide real-time feedback to the user.

These include:

- **OLED Display (SSD1306):** Displays system messages such as system initialization, waiting state, and access results.
- **Buzzer:** Provides audible alerts indicating access outcomes.
- **LED Indicator:** Visually indicates successful access events.

These components collectively simulate the behavior of a real access control system by providing immediate feedback to the user following an access attempt.

The sensing layer therefore serves as the entry point for access events into the IoT system.

2.4 Embedded / Edge Processing Layer

The embedded processing layer is responsible for interpreting sensor inputs, generating event data, and coordinating communication with external systems. In the SmartDoor system, this layer is implemented using the ESP32 microcontroller.

The ESP32 was selected due to its integrated WiFi capabilities, sufficient processing power for embedded IoT applications, and support for various hardware interfaces.

The responsibilities of the edge processing layer include:

- Monitoring input signals from access buttons
- Determining whether an access event is authorized or denied
- Generating structured event data
- Providing user feedback through LEDs, buzzer, and display
- Managing WiFi connectivity
- Synchronizing system time using Network Time Protocol (NTP)

- Transmitting access event data to cloud services

When an access event occurs, the ESP32 generates a structured JSON data payload containing relevant event information. This payload includes a unique device identifier, the status of the access event, and the timestamp of the event.

An example event record generated by the system is shown below:

```
{
  "device": "SmartDoorNode01",
  "status": "granted",
  "timestamp": "2026-03-08 22:13:28"
}
```

By performing initial processing at the edge device, the system reduces communication overhead and ensures that event data is properly structured before transmission to the cloud infrastructure.

2.5 Communication Layer

The communication layer enables data transmission between the embedded device and remote cloud infrastructure. In the SmartDoor system, wireless communication is achieved using the ESP32's built-in WiFi module.

Two communication protocols were implemented to demonstrate different IoT communication mechanisms.

2.5.1 HTTP REST Communication

The primary communication mechanism used for cloud data storage is the HTTP REST protocol. Access events generated by the ESP32 are transmitted to the Firebase Realtime Database using HTTP POST requests.

This approach allows the embedded system to interact directly with the cloud database through a RESTful API. Each access event is sent as a JSON payload and stored as a new record in the database.

The advantages of using HTTP communication include:

- simplicity of implementation
- compatibility with REST APIs
- reliable data transmission
- widespread support across platforms

HTTP communication therefore provides a reliable mechanism for storing access logs in the cloud database.

2.5.2 MQTT Messaging

In addition to HTTP communication, the system also publishes access events using the Message Queuing Telemetry Transport (MQTT) protocol.

MQTT is a lightweight publish-subscribe messaging protocol widely used in IoT systems for real-time data distribution. Access events are published to a public MQTT broker using a predefined topic.

This messaging mechanism allows other IoT systems or monitoring tools to subscribe to the access event stream and receive updates in real time.

The implementation of MQTT demonstrates the flexibility of the system in supporting multiple communication paradigms within an IoT environment.

2.6 Data Management Layer

The data management layer is responsible for storing, organizing, and retrieving access event data. In the SmartDoor system, this function is implemented using Firebase Realtime Database, a cloud-hosted NoSQL database platform.

Firebase provides several features that make it suitable for IoT data management, including:

- real-time data synchronization
- scalable cloud infrastructure

- JSON-based data storage
- REST API access for embedded devices
- secure access control mechanisms

Each access event generated by the ESP32 is stored as a structured record in the database. The database automatically assigns a unique identifier to each event entry, ensuring that all access events are preserved for future analysis.

The stored data forms the historical record used for monitoring system activity and generating analytical insights.

The database structure used in the project follows a hierarchical JSON format where each access event is stored as an individual entry under the access logs node.

2.7 Application Layer

The application layer provides the interface through which users interact with the monitoring system. In this project, the application layer is implemented using a Streamlit-based web dashboard.

The dashboard retrieves access data from the Firebase database and presents it through interactive visualizations and monitoring tools.

The application provides several monitoring capabilities including:

- a table displaying all recorded access logs
- graphical representation of authorized and denied access events
- timeline visualization of access activity
- daily access statistics
- automated detection of repeated unauthorized access attempts
- user authentication for secure dashboard access

The dashboard automatically refreshes at predefined intervals to ensure that newly recorded access events are displayed in near real-time. This application layer enables administrators or users to monitor access activity remotely and obtain insights into system usage patterns.

2.8 System Data Flow

The flow of information within the SmartDoor system follows a sequential process beginning with event detection and ending with data visualization.

The system data flow can be summarized as follows:

1. A user interaction triggers an access event through the sensing layer.
2. The ESP32 microcontroller detects the input signal.
3. The embedded system generates a structured access event record.
4. The event data is transmitted to the cloud database through the communication layer.
5. The Firebase database stores the event record.
6. The Streamlit dashboard retrieves stored data from Firebase.
7. The dashboard visualizes the access data through charts and tables.

This process ensures continuous monitoring of door access events and provides a centralized repository for historical data analysis.

2.9 Summary

This chapter presented the system design and architecture of the SmartDoor IoT monitoring system. The system was designed using a layered IoT architecture consisting of sensing, embedded processing, communication, data management, and application layers. Each layer performs a specific function that contributes to the overall operation of the monitoring system. The sensing layer captures access events, the embedded processing layer generates event data, the communication layer transmits data to the cloud, the data management layer stores access logs, and the application layer provides visualization and monitoring capabilities.

CHAPTER 3: SYSTEM IMPLEMENTATION

3.1 Introduction

This chapter describes the practical implementation of the SmartDoor IoT Access Monitoring System. The implementation phase involved integrating embedded hardware, communication protocols, cloud infrastructure, and a monitoring application into a unified IoT system. The system was developed as a functional prototype capable of detecting door access events, transmitting event data to a cloud database, and visualizing the collected information through a monitoring dashboard.

The implementation process followed the layered architecture described in Chapter 2. Each system layer was implemented using appropriate technologies and tools to ensure reliable sensing, communication, data storage, and visualization. The prototype was developed and tested using simulation tools and software frameworks that support embedded IoT development.

The main implementation components include the hardware setup, firmware development for the embedded device, cloud database integration, and the development of the monitoring dashboard.

3.2 Hardware Implementation

The hardware implementation forms the physical layer of the SmartDoor monitoring system. The hardware components were selected based on their suitability for IoT applications, ease of integration, and availability of software support.

The core hardware component used in this project is the ESP32 microcontroller, which serves as the embedded processing unit of the system. The ESP32 was selected due to its built-in WiFi connectivity, sufficient computational capabilities, and compatibility with the Arduino development framework.

The hardware components used in the prototype include:

- ESP32 microcontroller
- OLED display (SSD1306)
- Push buttons for simulated access events

- LED indicator
- Active buzzer

The push buttons simulate two types of access events: authorized access and unauthorized access. When a button is pressed, the ESP32 detects the input signal and interprets it as an access event.

The OLED display provides real-time feedback to the user by displaying system messages such as system initialization, waiting state, and access results. The buzzer generates audible alerts indicating whether access has been granted or denied, while the LED indicator provides visual confirmation of successful access attempts.

The hardware components are connected to the ESP32 through its General-Purpose Input/Output (GPIO) pins. The OLED display communicates with the microcontroller using the I2C interface, while the push buttons and output devices are connected through digital GPIO pins.

To simplify hardware testing and development, the entire system was simulated using the Wokwi IoT simulation platform, which allows developers to test embedded systems without requiring physical hardware components.

3.3 Firmware Development

The firmware of the SmartDoor system was developed using the Arduino framework, which provides a flexible and widely used platform for embedded system programming. The firmware controls the operation of the ESP32 microcontroller and manages interactions between hardware components, network connectivity, and cloud communication.

The firmware performs several critical functions including event detection, local feedback generation, network communication, and cloud data transmission.

3.3.1 Event Detection

The firmware continuously monitors the input pins connected to the push buttons. When a button press is detected, the system interprets it as an access event.

Two possible event states are defined:

- **Granted Access** – representing an authorized entry
- **Denied Access** – representing an unauthorized access attempt

Each event triggers corresponding feedback through the OLED display, buzzer, and LED indicator.

3.3.2 System Feedback

Once an access event is detected, the system immediately provides feedback to the user. The OLED display shows a message indicating whether access was granted or denied. At the same time, the buzzer produces distinct sound patterns for authorized and unauthorized access events.

This feedback mechanism ensures that users interacting with the system receive immediate confirmation of the access result.

3.3.3 Network Connectivity

The ESP32 connects to a wireless network using its built-in WiFi module. The firmware establishes a connection to the available network and maintains connectivity throughout system operation.

To ensure accurate event logging, the system synchronizes its internal clock using the Network Time Protocol (NTP). This allows each access event to be recorded with an accurate timestamp.

3.3.4 Data Payload Generation

Whenever an access event occurs, the firmware generates a structured data payload in JSON format. This payload contains information about the event and is transmitted to the cloud database.

A typical event payload generated by the system is shown below:

```
{
  "device": "SmartDoorNode01",
  "status": "granted",
  "timestamp": "2026-03-08 22:13:28"
}
```

This standardized structure ensures that the data can be easily stored and processed by the cloud platform.

3.4 Cloud Database Integration

The SmartDoor system uses Firebase Realtime Database as the cloud data storage platform. Firebase is a cloud-hosted NoSQL database that allows real-time storage and synchronization of JSON data.

The embedded device communicates with Firebase using HTTP REST requests. Whenever an access event occurs, the ESP32 sends an HTTP POST request containing the event payload to the Firebase database endpoint.

Once received, the database stores the event as a new entry under the access logs collection. Each entry is assigned a unique identifier automatically generated by the database.

The use of Firebase offers several advantages including:

- scalable data storage
- real-time synchronization
- easy integration with embedded systems
- support for RESTful communication

The stored data forms a centralized repository of access logs that can be retrieved for monitoring and analysis.

3.5 Monitoring Dashboard Implementation

To enable real-time monitoring and visualization of access events, a web-based dashboard was developed using the Streamlit framework. Streamlit is a Python-based framework that simplifies the development of data-driven web applications.

The dashboard retrieves access event data from the Firebase database using HTTP requests. The retrieved data is processed using the Pandas data analysis library and visualized using the Plotly visualization library.

The monitoring dashboard provides several functionalities including:

- real-time access log visualization
- graphical representation of authorized and denied access attempts
- timeline visualization of access events
- daily access statistics
- detection of repeated unauthorized access attempts
- user authentication for secure dashboard access

The dashboard automatically refreshes at regular intervals to ensure that newly recorded access events are displayed promptly.

The interface allows system administrators to observe system activity, analyze access patterns, and detect potential security threats.

3.6 System Integration

After completing the development of the hardware components, embedded firmware, cloud database configuration, and monitoring dashboard, the final stage of implementation involved integrating these subsystems into a fully operational IoT monitoring platform.

System integration ensured that each component could communicate correctly with the others and that the entire system functioned as a unified architecture. Particular attention was given to validating the interaction between the embedded device and the cloud infrastructure, as well as confirming that data stored in the database could be successfully retrieved and visualized through the monitoring dashboard.

The integration process involved the following steps:

1. Configuring the ESP32 firmware to establish a stable WiFi connection and synchronize system time using the Network Time Protocol (NTP).

2. Implementing HTTP-based communication between the embedded device and the Firebase Realtime Database to enable automatic logging of access events.
3. Verifying that each access event generated by the microcontroller was successfully transmitted and stored as a new entry in the cloud database.
4. Developing the Streamlit dashboard to retrieve stored access data through the Firebase REST API and transform the data into structured tables and visual analytics.
5. Testing the full system workflow by simulating multiple access events and confirming that the resulting records were correctly logged and displayed in the monitoring interface.

Through this integration process, the individual system components were successfully combined into a cohesive IoT monitoring system capable of capturing access events, transmitting them to the cloud infrastructure, and presenting the information through an interactive dashboard.

This stage confirmed that the proposed system architecture could operate reliably as a complete end-to-end IoT monitoring solution.

3.7 Summary

This chapter described the implementation of the SmartDoor IoT monitoring system. The system integrates embedded hardware components, firmware running on an ESP32 microcontroller, cloud-based data storage using Firebase Realtime Database, and a monitoring dashboard developed using the Streamlit framework.

The implementation demonstrates how embedded devices, wireless communication, cloud computing platforms, and visualization tools can be combined to build a functional IoT monitoring solution.

The next chapter presents the evaluation of the system, including performance analysis, system testing, and discussion of the obtained results.

CHAPTER 4: SYSTEM EVALUATION AND RESULTS

4.1 Introduction

This chapter presents the evaluation of the SmartDoor IoT Access Monitoring System. The objective of the evaluation was to verify the functionality, reliability, and responsiveness of the implemented system. System testing focused on validating the correct operation of the sensing components, communication mechanisms, cloud data storage, and the monitoring dashboard.

Performance metrics were selected based on the project guidelines and included system responsiveness, communication latency, data transmission reliability, and basic security monitoring capability.

4.2 System Functionality Testing

Functional testing was conducted to verify that the system operates as expected across all implemented layers. The tests focused on confirming that access events are correctly detected, processed by the embedded system, transmitted to the cloud database, and displayed on the monitoring dashboard.

Table 4.1 summarizes the primary functional tests performed during system validation.

Table 4.1: Functional System Tests

Test Case	Description	Expected Outcome	Result
Access Detection	Button press simulating access event	Event detected by ESP32	Successful
Feedback Response	System display and buzzer feedback	Correct feedback message displayed	Successful
Cloud Logging	Event transmitted to Firebase	Event stored in database	Successful
Dashboard Retrieval	Dashboard fetches stored logs	Data displayed in table	Successful

Data Visualization	Chart displays granted vs denied access	Chart updates correctly	Successful

The results indicate that the system successfully performs all intended functions.

4.3 Communication Latency

Communication latency refers to the time required for an access event to propagate from the sensing layer to the monitoring dashboard. This includes processing time within the embedded device, transmission time over the network, and database update time. Latency measurements were obtained by observing the time difference between an access event occurring and the corresponding update appearing on the Streamlit dashboard. The observed average latency ranged between 1–3 seconds, depending on network conditions and dashboard refresh intervals. This latency level is acceptable for monitoring applications where near real-time visibility of events is required.

4.4 Data Transmission Reliability

Data transmission reliability was evaluated by verifying that access events generated by the ESP32 were consistently recorded in the Firebase Realtime Database. Multiple access events were triggered sequentially, and the resulting records were compared with the number of generated events. The test results showed that all generated events were successfully transmitted and stored in the database without loss. The use of HTTP POST requests ensured reliable communication between the embedded device and the cloud platform. These results demonstrate that the system maintains reliable data logging under normal operating conditions.

4.5 System Responsiveness

System responsiveness refers to the time required for the embedded system to react to user input and provide immediate feedback.

When an access event was triggered, the system performed the following actions:

1. Detected the button input.
2. Displayed the corresponding access message on the OLED display.
3. Activated the buzzer and LED indicator.

The response from event detection to user feedback occurred almost instantaneously, typically within milliseconds, indicating that the embedded processing layer operates efficiently.

4.6 Security Event Monitoring

The monitoring dashboard includes a basic security detection mechanism designed to identify repeated unauthorized access attempts. The system evaluates recent access logs and generates a security alert when multiple denied access events occur within a short time window. Testing involved generating several unauthorized access events consecutively. The dashboard successfully detected this pattern and displayed a warning message indicating a potential security issue. This functionality demonstrates the capability of the system to provide basic anomaly detection for security monitoring.

4.7 Discussion of Results

The evaluation results demonstrate that the SmartDoor IoT monitoring system successfully fulfills its design objectives. The system reliably detects access events, transmits event data to the cloud database, and visualizes the collected information through the monitoring dashboard. The integration of embedded hardware, cloud infrastructure, and a visualization platform provides a complete end-to-end IoT monitoring solution. Communication latency remained within acceptable limits for real-time monitoring applications, while the reliability tests confirmed that access events were consistently logged without data loss. Furthermore, the implemented dashboard provides

meaningful insights into system usage patterns and enables the detection of suspicious access activity.

4.8 Summary

This chapter evaluated the performance of the SmartDoor IoT Access Monitoring System through functional testing and performance analysis. The results confirm that the system operates reliably across all architectural layers and successfully demonstrates the capabilities of an IoT-based monitoring solution.

CHAPTER 5: LIMITATIONS, CONCLUSIONS, AND RECOMMENDATIONS

5.1 Introduction

This chapter summarizes the key findings of the project, outlines the limitations of the implemented system, and provides recommendations for future improvements. The SmartDoor IoT Access Monitoring System demonstrates the practical integration of embedded hardware, wireless communication, cloud data storage, and dashboard-based monitoring within a layered IoT architecture.

5.2 System Limitations

Although the implemented system successfully demonstrates the core functionality of an IoT-based access monitoring platform, several limitations were identified during development and testing.

First, the system was implemented primarily as a prototype using simulated access inputs. Access events were generated through push buttons rather than real authentication mechanisms such as RFID cards, biometric systems, or keypad-based entry. As a result, the prototype focuses on monitoring functionality rather than full authentication control.

Second, the system relies on a stable wireless network connection for communication between the embedded device and the cloud database. Network interruptions may delay event transmission or temporarily affect real-time monitoring capabilities.

Third, the current implementation uses basic authentication for dashboard access. More advanced access control mechanisms such as multi-user authentication and role-based permissions were not implemented in this prototype.

Finally, secure communication mechanisms such as encrypted data transmission or device authentication were not incorporated into the system. These security measures would be necessary for deployment in real-world environments.

5.3 Conclusion

The objective of this project was to design and implement a smart IoT-based door access monitoring system capable of capturing access events, transmitting the data to a cloud database, and visualizing the information through a monitoring dashboard.

The results demonstrate that the proposed system successfully integrates sensing components, embedded processing, wireless communication, cloud data storage, and a web-based monitoring interface. The ESP32 microcontroller effectively detects access events and transmits structured event data to the Firebase Realtime Database. The Streamlit dashboard retrieves this data and presents it through graphical visualizations and access logs that allow users to monitor system activity in near real time.

The evaluation results confirmed that the system operates reliably across all architectural layers. Access events were consistently detected, transmitted, stored, and visualized through the monitoring interface. The implemented dashboard also demonstrated basic analytical capabilities, including access statistics and detection of repeated unauthorized access attempts.

Overall, the SmartDoor system illustrates how IoT technologies can be used to build practical monitoring solutions that improve visibility, record system activity, and support security monitoring applications.

5.4 Recommendations for Future Work

Several improvements could enhance the functionality and security of the proposed system.

Future implementations may integrate real authentication mechanisms such as RFID readers, biometric sensors, or keypad-based authentication devices to provide a complete access control solution. Security features could also be improved by implementing encrypted communication protocols, device authentication, and role-based user access control for the monitoring dashboard.

Additional analytical capabilities such as machine learning–based anomaly detection could further enhance the system by automatically identifying unusual access patterns. Finally, the system could be extended with mobile applications or notification systems that provide real-time alerts when suspicious access activity is detected.